

## Table of Contents

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>1) LEARNING OBJECTIVES.....</b>                          | <b>3</b> |
| <b>2) INTRODUCTION.....</b>                                 | <b>3</b> |
| <b>3) TARGET LABORATORY SOURCE CODE.....</b>                | <b>4</b> |
| <b>4) LABORATORY PROCEDURES AND TASKS.....</b>              | <b>4</b> |
| a) Task A: Building with Debugging Symbols.....             | 4        |
| b) Task B: Setting Breakpoints and Runtime Positioning..... | 4        |
| i) Verification Questions for Task B.....                   | 5        |
| c) Task C: Mapping Static Text and Data Section Bounds..... | 5        |
| i) Verification Questions for Task C.....                   | 6        |
| d) Task D: Auditing Initialized Data Memory Layouts.....    | 7        |
| e) Task E: Workspace Demobilization.....                    | 7        |
| <b>5) DELIVERABLES AND LAB REPORT REQUIREMENTS.....</b>     | <b>8</b> |
| <b>6) POST-LAB TECHNICAL ASSESSMENT QUESTIONS.....</b>      | <b>8</b> |

## List of Figures

|                                                                                                       |   |
|-------------------------------------------------------------------------------------------------------|---|
| Listing 1. Target Mapping and Verification Code for Text and Data Segments.....                       | 4 |
| Listing 2: Compiling demo2.c.....                                                                     | 4 |
| Listing 3: Setting breakpoint at line 22 and hitting it in GDB.....                                   | 5 |
| Listing 4: GDB info files output of demo2 showing the section address ranges.....                     | 6 |
| Listing 5:GDB output showing the address and value of data_global_var and the string literal<br>..... | 7 |
| Listing 6: Terminating gdb.....                                                                       | 7 |

# Auditing of Text and Initialized Data Segments via Hardware Disassembly

## 1) LEARNING OBJECTIVES

By completing this laboratory assignment, students will be able to:

- a) Evaluate the structural composition and isolation boundaries separating the Text segment (.text, .rodata) from the Initialized Data segment (.data) on your system architectures.
- b) Disassemble compiled application subroutines into machine opcodes to track hardware instruction pointers.

## 2) INTRODUCTION

An application container layout under execution divides instructions from states. The Text segment (.text) stores the raw, read-only hardware machine code generated from programming loops. To safeguard binary execution files from malicious manipulation during runtime windows, modern operating systems flag the virtual memory pages assigned to the text boundaries as non-writable. Read-only constants and string patterns are pulled into an adjacent sub-compartment termed the .rodata segment.

Conversely, global variables or context states that contain explicit initialized parameters at build time are placed into the Read-Write Data segment (.data). To build low-level systems capable of safe instruction execution, systems architects must inspect text structures directly via functional disassemblers and map out exactly how static memory arrays align with physical hardware components.

### 3) TARGET LABORATORY SOURCE CODE

Students must replicate the architectural code implementation listed in Listing 1. Save this implementation locally on your Linux machine as demo2.c.

```
GNU nano 8.7.1 demo2.c
#include <stdio.h>

// 1. Data Section (Initialized global data)
int data_global_var = 0x12345678;

// 2. RoData Section (Read-only string constant literal)
const char *text_string_literal = " Text_Segment_Verification";

// 3. Text Section (Function execution code block)
void check_execution() {
    printf("Executing within the text segment instruction block.\n");
}

int main() {
    printf("Addresses:\n");
    printf("Data (Global Initialized) : %p\n", (void*)&data_global_var);
    printf("Text String (RoData): %p\n", (void*)text_string_literal);
    printf("Text Code (Function) : %p\n", (void*)&check_execution);

    check_execution();

    // Breakpoint here to inspect text and data layers
    printf("Ready for Text/Data segment inspection!\n");
    return 0;
}
```

*Listing 1. Target Mapping and Verification Code for Text and Data Segments*

### 4) LABORATORY PROCEDURES AND TASKS

#### a) Task A: Building with Debugging Symbols

Compile the source code inside your terminal application using the -g flag to preserve structural naming symbols and debugging metadata maps:

```
bipin@bipin-VirtualBox:~$ gcc -g demo2.c -o demo2
```

*Listing 2: Compiling demo2.c*

## b) Task B: Setting Breakpoints and Runtime Positioning

Initialize the target binary within the debugger environment and configure execution anchors to suspend operations while the initialized structures remain active

```
bipin@bipin-VirtualBox:~$ gdb ./demo2
```

Configure a breakpoint at line 22 to halt operations right before the application returns control back to the operating system

shell:

```
(gdb) break 22
Breakpoint 1 at 0x11f9: file demo2.c, line 23.
(gdb) run
Starting program: /home/bipin/demo2
```

```
Breakpoint 1 at 0x11f9: file demo2.c, line 23.
(gdb) run
Starting program: /home/bipin/demo2

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Addresses:
Data (Global Initialized) : 0x555555558010
Text String (ROData): 0x555555556008
Text Code (Function) : 0x555555555169
Executing within the text segment instruction block.

Breakpoint 1, main () at demo2.c:23
23      printf("Ready for Text/Data segment inspection!\n");
```

*Listing 3: Setting breakpoint at line 22 and hitting it in GDB*

## c) Task C: Mapping Static Text and Data Section Bounds

Query the debugger to print the file section layout map structured by the linker:

```
(gdb) info files
Symbols from "/home/bipin/demo2".
Native process:
  Using the running image of child process 11559.
  While running this, GDB does not access memory from...
Local exec file:
  '/home/bipin/demo2', file type elf64-x86-64.
  Entry point: 0x555555555080
  0x0000555555554350 - 0x0000555555554374 is .note.gnu.build-id
  0x0000555555554374 - 0x0000555555554390 is .interp
  0x0000555555554390 - 0x00005555555543b4 is .gnu.hash
  0x00005555555543b8 - 0x0000555555554478 is .dynsym
  0x0000555555554478 - 0x000055555555450c is .dynstr
  0x000055555555450c - 0x000055555555451c is .gnu.version
  0x0000555555554520 - 0x0000555555554550 is .gnu.version_r
  0x0000555555554550 - 0x0000555555554628 is .rela.dyn
  0x0000555555554628 - 0x0000555555554658 is .rela.plt
  0x0000555555555000 - 0x000055555555501b is .init
  0x0000555555555020 - 0x0000555555555050 is .plt
  0x0000555555555050 - 0x0000555555555060 is .plt.got
  0x0000555555555060 - 0x0000555555555080 is .plt.sec
  0x0000555555555080 - 0x000055555555520f is .text
  0x0000555555555210 - 0x000055555555521d is .fini
  0x0000555555555600 - 0x000055555555560e8 is .rodata
  0x000055555555560e8 - 0x00005555555556124 is .eh_frame_hdr
  0x00005555555556128 - 0x000055555555561f4 is .eh_frame
  0x000055555555561f8 - 0x00005555555556228 is .note.gnu.property
  0x00005555555556228 - 0x00005555555556248 is .note.ABI-tag
  0x00005555555557db0 - 0x00005555555557db8 is .init_array
  0x00005555555557db8 - 0x00005555555557dc0 is .fini_array
  0x00005555555557dc0 - 0x00005555555557fb0 is .dynamic
  0x00005555555557fb0 - 0x00005555555558000 is .got
  0x00005555555558000 - 0x00005555555558020 is .data
  0x00005555555558020 - 0x00005555555558028 is .bss
  0x00007ffff7fc02a8 - 0x00007ffff7fc02cc is .note.gnu.build-id in /lib64/ld-linux-x86-64.so.2
  0x00007ffff7fc02d0 - 0x00007ffff7fc0408 is .hash in /lib64/ld-linux-x86-64.so.2
  0x00007ffff7fc0408 - 0x00007ffff7fc0564 is .gnu.hash in /lib64/ld-linux-x86-64.so.2
  0x00007ffff7fc0568 - 0x00007ffff7fc0910 is .dynsym in /lib64/ld-linux-x86-64.so.2
  0x00007ffff7fc0910 - 0x00007ffff7fc0bc1 is .dynstr in /lib64/ld-linux-x86-64.so.2
```

*Listing 4: GDB info files output of demo2 showing the section address ranges*

### i) Verification Questions for Task C

- **What information does the file type descriptor elf64-..... reveal about the hardware platform's structural address sizing and byte serialization constraints?**  
elf64-x86-64 means 64-bit addresses (8-byte pointers) on the x86-64 instruction set, which is always little-endian, so 0x12345678 is stored as 78 56 34 12.
- **Extract the boundary limits allocated to the .text section. Does the runtime address of check\_execution fit inside this map range?**  
The .text section runs from 0x1080 to 0x120f. The function check\_execution is at an address like 0x11xx, which is between these two numbers, so yes, it fits inside.

- **Locate the boundary space mapped to the .rodata segment. Verify mathematically if the string literal pointer address falls within this section.**

The .rodata section runs from 0x2000 to 0x20e8, and the string address (like 0x2004) is bigger than 0x2000 and smaller than 0x20e8, so it falls inside.

- **Locate the boundary parameters allocated to the .data section. Does the global variable address fit within this sector map?**

The .data section runs from 0x4000 to 0x4020, and the global variable address (like 0x4010) is between them, so yes, it fits inside.

- **Analyze the relative positions of the .data and .bss sections. Which segment occupies the lower memory range?**

The .data section (0x4000 to 0x4020) comes first, and .bss (0x4020 to 0x4028) starts right after it, so .data occupies the lower memory range

#### d) Task D: Auditing Initialized Data Memory Layouts

Examine the active literal data word stored inside your global variable. Inspect the memory layout address directly to determine raw data serialization representations:

```
(gdb) print &data_global_var
$1 = (int *) 0x55555558010 <data_global_var>
(gdb) x/wx &data_global_var
0x55555558010 <data_global_var>:      0x12345678
(gdb) print text_string_literal
$2 = 0x55555556008 " Text_Segment_Verification"
(gdb) x/s text_string_literal
0x55555556008: " Text_Segment_Verification"
(gdb) print &text_string_literal
$3 = (const char **) 0x55555558018 <text_string_literal>
```

*Listing 5: GDB output showing the address and value of data\_global\_var and the string literal*

## e) Task E: Workspace Demobilization

Terminate the runtime environment cleanly when completed:

```
(gdb) print &string_literal
$6 = (const char **) 0x555555558018 <text_string_literal>
(gdb) continue
Continuing.
Ready for Text/Data segment inspection!
[Inferior 1 (process 11559) exited normally]
(gdb) quit
bipin@bipin-VirtualBox:~$
```

*Listing 6: Terminating gdb*

## 5) DELIVERABLES AND LAB REPORT REQUIREMENTS

You must compile a formal laboratory writeup documenting their results and satisfying the following analysis directives:

- Provide full terminal log captures or screenshots confirming successful compilation and step execution parameters.
- Populate a structured matrix displaying the recorded start and stop virtual addresses for the .text, .rodata, and .data segments extracted during your live debugging sessions.
- Document the raw hex output string returned during Task D when parsing the global variable. Contrast the written order of bytes in your C code source configuration file (0x12345678) against the layout printed by the memory examiner.

## 6) POST-LAB TECHNICAL ASSESSMENT QUESTIONS

**i) Review your results from Task D regarding byte order. Does your system target execute operations under a Little-Endian serialization style or a Big-Endian serialization style? Justify your classification using the raw hex results collected during verification.**

- The system is Little-Endian, because x86-64 stores the least significant byte first, so 0x12345678 is kept in memory as 78 56 34 12 (a Big-Endian system would store 12 34 56 78).

**ii) Why does the operating system isolate the compiled code instructions (.text) from literal constant tracking string chains (.rodata) if both sections are policed as read-only pages? Discuss the concepts of instruction caches (I-Cache) and memory paging access profiles**

- Code and constants are used in different ways, so the OS keeps them apart. Code needs permission to run, but constants should only be read, never run. The CPU also uses one cache for code (ICache) and another for data, so keeping each kind together makes the computer faster and safer